

PROJECT

Programming Language Paradigms

Higher-Order Functions and Currying
in Python, Kotlin and Scala

Worked by:

Viola Zeneli

Sidrit Rrustemi

Denisa Korra

Accepted by: Dr. Erind Bedalli & M. Sc. Griselda Alushllari

2024 – 2025

TABLE OF CONTENTS

Scope of the Project 2
Work Division 2
Introduction to Higher-Order Functions and Currying 3
Python: HOFs & Currying 6
Kotlin: HOFs & Currying 18
Scala: HOFs & Currying 26
Language Comparisons 33
Summary 35

Scope of the Project

This project aims to explore and compare how Higher-Order Functions (HOFs) and Currying are defined, implemented, and used in three modern programming languages: Python, Kotlin, and Scala. These two functional programming concepts are central to writing clean, modular, and reusable code. The project provides a theoretical foundation for both HOFs and currying, followed by detailed, language-specific implementations and examples for each.

Each group member focused on one language to analyze:

- How higher-order functions are used to treat functions as first-class citizens
- The syntax and mechanism for currying and partial application
- Built-in HOFs like map, filter, reduce, and custom HOFs
- The use of anonymous (lambda) functions with HOFs
- Comparisons between HOFs and currying with code and real-world use cases

Additionally, the project includes structured documentation in Word and PowerPoint formats, a summary comparison table, curried and partially applied function examples, and organized source code for all three languages. The final result is a cross-language study that not only explains the core concepts but also demonstrates how Python, Kotlin, and Scala each embrace functional programming principles in practical and distinct ways.

Work Division

Higher-Order Functions and Currying in Python → Denisa Korra
Higher-Order Functions and Currying in Kotlin → Viola Zeneli
Higher-Order Functions and Currying in Scala → Sidrit Rrustemi

Introduction to Higher-Order Functions and Currying

What Are Higher-Order Functions?

A higher-order function is a function that:

- Takes one or more functions as arguments.
- Returns another function as its result.

These functions are a powerful feature in functional programming languages and enable a high level of abstraction.

1. **Code Reusability:** Higher-order functions allow developers to write more modular and reusable code by abstracting common patterns of computation.
2. **Generic Behavior:** They enable the creation of generic functions that can operate on various data or behaviors, simply by passing different functions as parameters.
3. **Example** - map: A classic example is the map function, which takes a function and a list, then applies the function to each element of the list, returning a new list of results.

What is Currying?

Currying is a functional programming technique that transforms a function with multiple arguments into a sequence of nested functions, each taking a single argument.

For example, a function defined as add(a, b) – which takes two arguments - can be transformed into add(a)(b). In this curried form, the first function takes argument a and returns another function that takes argument b.

A. Currying transforms a multi-parameter function into a chain of single-parameter functions. This means you apply arguments one at a time rather than all at once.

B. It allows for partial application—providing some arguments up front and deferring others. This helps create specialized versions of functions tailored to particular use cases.

C. Currying improves code reusability, encourages functional composition, and often results in cleaner, more declarative code.

D. It is especially useful in languages that treat functions as first-class citizens, allowing developers to build highly flexible and composable code structures.

Why Higher-Order Functions and Currying Matter in Functional Programming and in Modern Languages

Functional Programming

- **Reusability**:
  Higher-order functions allow us to abstract not just values, but behaviors. By passing functions as parameters, we can reuse general-purpose logic across different contexts, reducing code duplication and increasing maintainability.

- **Modularity**:
  Functions can be composed together to form more complex operations. This encourages writing small, focused, and well-defined pieces of logic that work seamlessly together—resulting in cleaner, more modular codebases.

- **Flexibility through Currying**:
  Currying breaks down a function that takes multiple arguments into a sequence of functions that each take one argument. This enables **partial application**—where some arguments are fixed early—resulting in flexible and adaptable code. It allows developers to define specialized versions of functions by pre-filling certain parameters.

- **Encapsulation of Behavior**:
  Higher-order functions allow you to encapsulate customizable behavior. For example, you can define a general-purpose function for iterating over data and pass in different logic for filtering, mapping, or reducing values, depending on your needs. This creates a clear separation between **control flow** and **business logic**.

- **Real-World Examples**:
  Common functions like `map`, `filter`, and `reduce` in functional programming are built using higher-order functions. These tools operate on lists and collections and are essential for transforming, filtering, and summarizing data in a concise and expressive way.

Modern Languages

Partial Application

Partial application is a functional programming technique that involves fixing a few arguments of a function and producing a new function with fewer arguments. This is made possible through currying, where a multi-parameter function is transformed into a sequence of functions, each taking one argument.

- **Function Specialization**:
  It allows developers to create specialized versions of general functions by pre-filling common arguments. For example, if a function calculates tax based on region and income, partially applying the region can yield a reusable function for that region.

- **Immutability and Predictability**:
  Since partially applied functions don't alter state, they promote immutability, leading to fewer side effects and more predictable code behavior.

- **Enhanced Composition**:
  Partially applied functions integrate smoothly into function pipelines, allowing the creation of complex behavior from simple, modular components. This makes the codebase more maintainable and easier to extend.

Enhanced Readability

Currying improves how functions are structured and understood, contributing to more readable and maintainable code.

- **Step-by-Step Logic**:
  Instead of passing all arguments at once, currying lets us handle them one by one. This makes the flow of data clearer and each transformation step more visible.

- **Flexible and Clear Syntax**:
  Currying allows you to write functions that are cleaner and more intuitive. You can invoke them progressively, which helps when building dynamic pipelines or reusing logic in multiple places.

- **Improved Maintainability**:
  With better function names and reduced redundancy through partial application, the code becomes self-explanatory. This aids in both collaborative development and long-term maintenance.

Python: HOFs & Currying

HOFs in Python

Functions in Python are first-class citizens, meaning they can be:

- **Assigned to variables.**

  You can assign a function to a variable, which essentially gives the function a new name or reference. This allows for flexible naming, reuse, and easier composition of behavior.

- **Passed as arguments to other functions.**

  One of the most powerful features of first-class functions is that you can pass them as arguments to other functions. This enables the creation of higher-order functions—functions that can operate on or return other functions.

- **Returned from other functions.**

  Python also allows functions to be returned from other functions. This is a key aspect of building closures and supports advanced functional patterns such as currying and decorators.

```python
def apply_function(func, value):  1 usage
    return func(value)

def square(x):  1 usage
    return x * x

result = apply_function(square,  value: 4)

print(result)

```

```
16

Process finished with exit code 0
```

Currying in Python

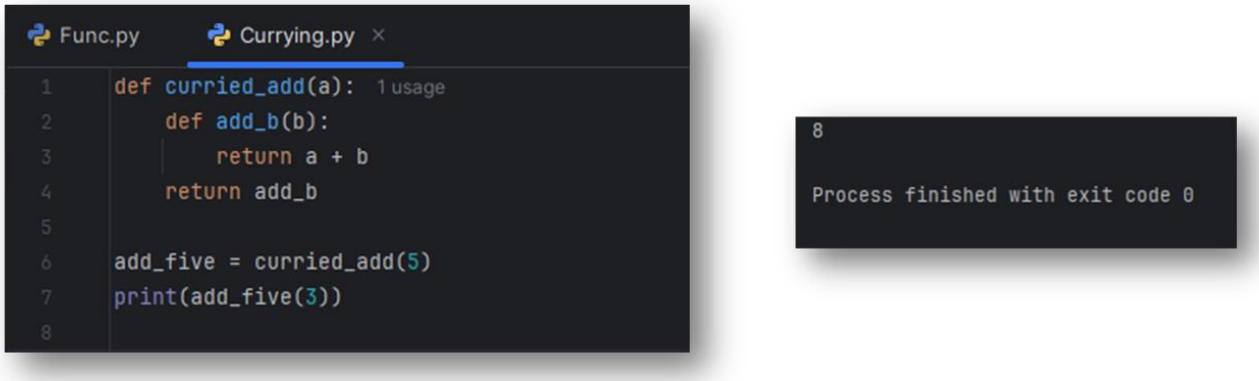
Python does not support currying natively in the same way languages like Haskell do. However, Python makes it possible to simulate currying behavior using closures or libraries like functools.partial.

Using Closures to Simulate Currying

A closure in Python is a function object that remembers values in enclosing scopes even if those scopes are no longer active. This makes it an effective way to simulate currying.

How Closures Help Currying

Closures allow the creation of functions that remember the arguments from previous calls, enabling step-by-step argument passing like currying does.



Using functools.partial for Partial Application

While not true currying, the functools module allows partial application of functions.

Differences between HOFs and Currying

Concept	Higher-Order Functions (HOFs)	Currying
How It Works	You can pass functions as arguments or return them from other functions.	You create a sequence of functions, each accepting one argument and returning another function.
Purpose	To abstract and manipulate behavior by passing functions around.	To partially apply arguments to a function step-by-step.
Use Case	Used for function composition, abstraction, and higher-level operations like map(), filter(), reduce().	Used when you want to pre-set one or more arguments of a function and apply the remaining arguments later.
Real-World Example	- Event handlers in GUI programming where behavior is passed as a function. - Functional libraries like map(), filter(), reduce() which take functions as arguments.	- Logging where a part of the function (like a log level) is set, and other values (like message) are passed later. - Mathematical operations like add(a)(b) where a is fixed first, and b is added later.
When to Use	When you need flexibility, reuse code, or compose multiple operations into a single function.	When you want to reuse a function with some arguments pre-set, making the function more specialized for future use.

When and Why Each is Used in Real-World Code

HOFs in Real-World Code

Map, Filter, and Reduce:

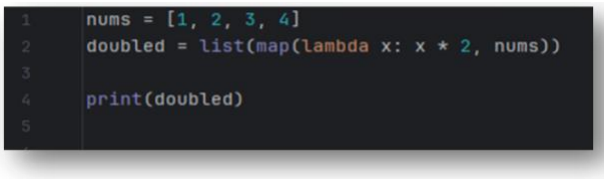
•These functions are built-in higher-order functions in Python. They take functions as arguments and return the result of applying that function to an iterable (list, tuple, etc.).

•They allow concise and functional programming, where logic can be abstracted and reused.

•You can pass custom functions into **map()**, **filter()**, or **reduce()** to apply specific behaviors on data collections.

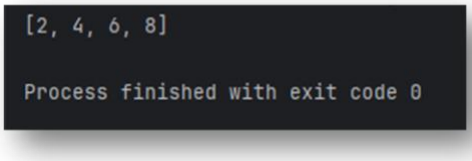
❖ Map()

The map() function applies a given function to each item in an iterable and returns a new iterable (list, tuple) of the results.



```
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, nums))

print(doubled)
```



```
[2, 4, 6, 8]

Process finished with exit code 0
```

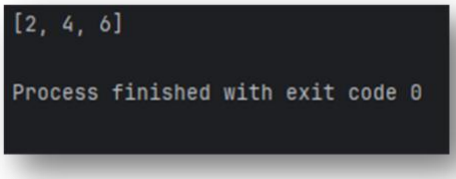
❖ Filter()

The filter() function filters out elements from an iterable based on a condition defined by a function. It returns a new iterable (filtered list) containing only the elements that satisfy the given condition.



```
nums = [1, 2, 3, 4, 5, 6]
even_nums = list(filter(lambda x: x % 2 == 0, nums))

print(even_nums)
```

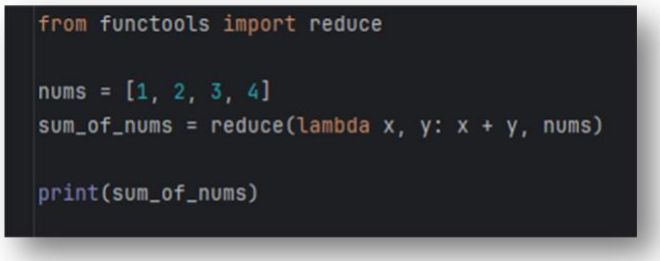


```
[2, 4, 6]

Process finished with exit code 0
```

❖ Reduce()

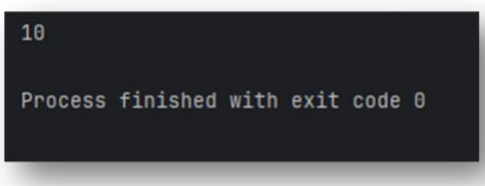
The reduce() function reduces an iterable (list, tuple) to a single value by applying a function that takes two arguments, combining them into one. It keeps applying the function cumulatively to the elements of the iterable.



```
from functools import reduce

nums = [1, 2, 3, 4]
sum_of_nums = reduce(lambda x, y: x + y, nums)

print(sum_of_nums)
```



```
10

Process finished with exit code 0
```

Lambda Functions in Python

In Python, a lambda function is a small, anonymous function defined using the `lambda` keyword. Lambda functions are used for creating short, throwaway functions without having to formally define them using `def`.

Syntax

The basic syntax of a lambda function is:

```
lambda arguments: expression
```

It can take any number of arguments but only one expression. The result of the expression is automatically returned.

Example

```
add = lambda a, b: a + b
print(add(3, 5))  # Output: 8
```

Common Use Cases

- Used with higher-order functions such as `map()`, `filter()`, and `reduce()`.
- Used for quick, inline operations where defining a full function is unnecessary.
- Common in GUI programming, data processing, and function chaining.

Lambda with map, filter, reduce

```
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x ** 2, numbers))
evens = list(filter(lambda x: x % 2 == 0, numbers))
# Output: [1, 4, 9, 16, 25] and [2, 4]
```

Limitations

- Lambda functions are limited to a single expression.
- They cannot contain statements, annotations, or documentation strings.
- For complex operations, using `def` is recommended for clarity and maintainability.

Custom Higher-Order Function

Function that takes another function as an argument

This is useful when the exact behavior of a function needs to be determined at runtime or reused in multiple scenarios.

```python
def function(func, value):  1 usage
    return func(value)

def double(x):  1 usage
    return x * 2

print(function(double,  value: 3))
```

```
6

Process finished with exit code 0
```

Function that returns another function

This allows for dynamic function creation, often used in decorators, closures, and factory functions.

```python
def multiplier(factor):  1 usage
    def multiply_by(x):
        return x * factor
    return multiply_by

multiply_by_2 = multiplier(2)
print(multiply_by_2(5))
```

```
10

Process finished with exit code 0
```

Partial Application in Python

In Python, partial application can be achieved using the functools.partial() function, which is a built-in utility for fixing a certain number of arguments of a function, creating a new function that takes fewer arguments.

How It Works:
Partial application means applying a function to a subset of its arguments.
Instead of repeatedly passing the same argument every time you call a function, you can "fix" that argument and generate a new function that only requires the remaining arguments.

Why Use Partial Application?

- **Cleaner code:** Reduces redundancy when the same argument is used repeatedly.
- **Flexible abstractions:** Allows creating more specialized or domain-specific versions of generic functions.
- **Useful in GUIs or callbacks:** Pre-set values for button clicks, network calls, etc.

```python
from functools import partial

def power(base, exponent):  1 usage
    return base ** exponent

square = partial(power, exponent=2)
print(square(5))
```

```
25

Process finished with exit code 0
```

Higher-Order Function with Function Composition

In functional programming, function composition refers to combining two or more functions to produce a new function. The result of one function becomes the input of the next. Python supports this powerful concept using higher-order functions (HOFs), which take functions as inputs and/or return new functions.

Why Use Composition in Python?

- **Improves clarity**: Break complex logic into smaller, testable functions.
- **Reduces duplication**: Combine reusable behaviors without rewriting logic.
- **Functional style**: Enables more declarative, readable code similar to math.

```
# Higher-Order Function Example with Function Composition
def compose(func1, func2):  1 usage
    return lambda x: func2(func1(x))  # Apply func1 first, then func2

# Two simple functions to manipulate strings
def to_uppercase(text):  1 usage
    return text.upper()

def add_exclamation(text):  1 usage
    return text + "!"

# Using the HOF to compose functions
shout = compose(to_uppercase, add_exclamation)
result = shout("hello")
print(result)
```

```
HELLO!

Process finished with exit code 0
```

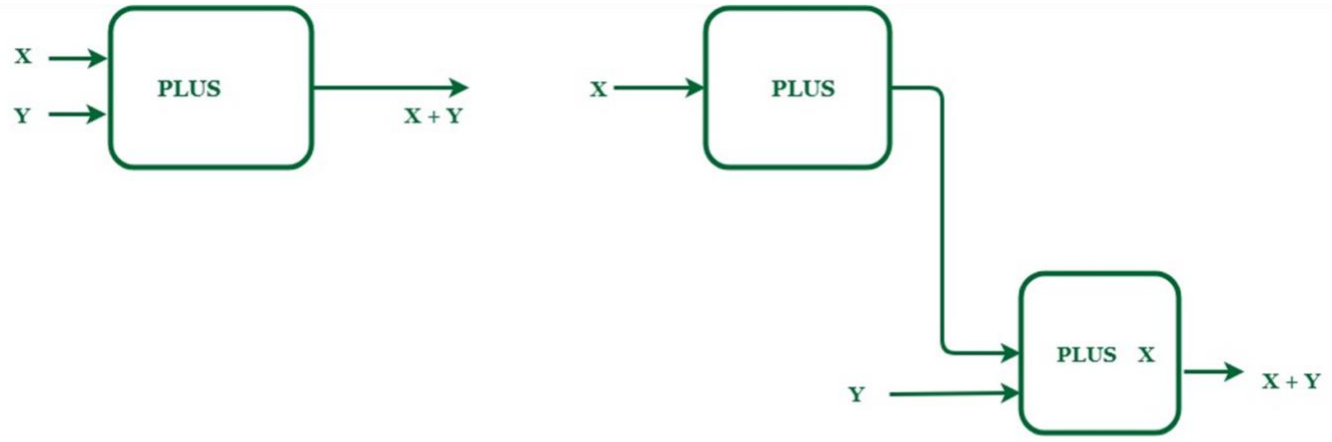
Currying Example with Custom Math Operation

Although Python doesn't provide native currying, you can simulate it using closures. With nested functions, it's possible to create elegant, curried solutions to real-world problems like math operations. This technique reinforces functional thinking and enables flexible, reusable code structures.

```python
# Currying Example for Mathematical Operations
def curried_math(op):  3 usages
    def operate(a):
        def apply_operation(b):
            if op == 'add':
                return a + b
            elif op == 'subtract':
                return a - b
            elif op == 'multiply':
                return a * b
            else:
                return None
        return operate

# Create curried functions for specific operations
add_five = curried_math('add')(5)
subtract_three = curried_math('subtract')(3)
multiply_by_two = curried_math('multiply')(2)

# Apply the operations
print(add_five(3))  # Output: 8
print(subtract_three(10))  # Output: 7
print(multiply_by_two(4))  # Output: 8
```



Normal Implementon of Function *Implemention of Curried Function*

Let's take an example, PLUS is a function which adds two numberWe wish to add two numbers X and Y. X will be the input to PLUS function which returns a function named PLUS X.
PLUS X function takes one number and add X to it. Now input to this function will be Y. Final output will be X + Y.

Advantages and disadvantages of higher ordered functions in python

Advantages	Disadvantages
Code Reusability: HOFs allow you to create generic functions that can be reused with different behavior by passing different functions.	**Complexity**: HOFs can introduce complexity and make code harder to understand, especially for beginners. It can be difficult to track the flow of execution when functions are passed around.
Modularity: HOFs allow for modular code by abstracting behavior into functions. You can pass different functions to achieve different behaviors without changing the structure.	**Performance Overhead:** Using HOFs, especially with functions like map(), filter(), or reduce(), can add extra overhead when compared to using simple loops, especially in tight loops or performance-sensitive applications.
Function Composition: HOFs facilitate function composition, where you can combine small functions to create complex operations.	**Debugging Difficulty**: Debugging code with HOFs can be more challenging because functions are passed around dynamically, which can make it harder to set breakpoints or track values during execution.
Functional Programming Paradigm: They promote functional programming, making Python code more flexible, reusable, and easier to maintain.	**Can Lead to Nested Functions:** Overuse of HOFs can lead to deeply nested functions or callbacks, making the code harder to follow and manage.
Cleaner Code: HOFs allow you to abstract complex logic and encapsulate it in functions, leading to cleaner, more readable code.	**Learning Curve:** Python developers accustomed to imperative programming may find it difficult to understand and use HOFs effectively.
Higher Flexibility: HOFs provide more flexibility and customization because they allow you to pass different functions to modify behavior without changing the structure.	**Limited Support for Side Effects:** Since functional programming (and HOFs) encourages pure functions (no side effects), certain real-world scenarios where side effects are needed (like modifying external states) can be more difficult to handle.
Encourages Cleaner Abstractions: HOFs can help abstract repetitive logic, reducing code duplication and allowing you to focus on higher-level problem-solving.	**Can Obscure Logic:** Overuse of HOFs can make the code harder to reason about, especially when combined with anonymous functions (lambdas). It might obscure the main logic of the program.

Advantages and disadvantages of currying in python

Advantages	Disadvantages
Partial Application: Currying allows for partial application, where you can fix some arguments of a function and apply the rest later. This makes it easier to create specialized functions.	**More Complex Syntax**: Currying requires a more complex syntax compared to regular functions. In Python, simulating currying can introduce nested functions, which may be harder to follow for beginners.
Reusability: Currying allows you to create more reusable functions by pre-setting some arguments. You can use the curried function with different remaining arguments, making it highly flexible.	**Increased Overhead**: Simulating currying in Python through closures or nested functions can introduce additional memory overhead and potentially slow down performance, especially if done extensively.
More Modular Code: It promotes a more modular approach by breaking down a function into smaller, focused functions. Each curried function can represent a part of the operation, improving clarity.	**Difficult to Debug**: Debugging curried functions can be challenging, especially when working with closures. Tracking variable values across nested functions can become complicated.
Clearer Function Composition: Currying makes it easier to compose functions together, leading to cleaner and more expressive code. You can create a pipeline of operations by currying.	**Not Native to Python**: Currying is not natively supported in Python. You have to simulate it using closures or libraries like functools.partial, making it less intuitive than in other languages like Haskell.
Increased Flexibility: You can create customized versions of functions by fixing arguments early. This leads to more flexible code where specific behavior can be set up upfront.	**Can Lead to Overuse**: Overuse of currying can lead to overly complex function signatures or too many small functions, making the code more fragmented and harder to manage.
Makes Code More Readable in Some Cases: In scenarios like logging, math operations, or configuration settings, currying helps make the code more readable by pre-setting common arguments.	**Limited to Simple Functions**: Currying is best suited for simple functions that can be split into unary functions. For more complex functions, currying may not be the most practical solution.
Supports Functional Programming Style: Currying encourages functional programming principles in Python, allowing you to use concepts like higher-order functions and function composition.	**Reduces Performance in Some Cases**: When currying is used with deeply nested functions or closures, it can result in slower performance due to the additional function calls and memory consumption.

Kotlin: HOFs & Currying

HOFs in Kotlin

Why are important?

- Kotlin is a multi-paradigm language, but embraces functional programming. HOFs make code more concise and composable.
- Using HOFs allows reusing logic with different behaviors and reducing code repetition.
- You can pass in your own function or lambda to dynamically control behavior(e.g., sorting, filtering, reducing).

Kotlin Syntax Support

EXAMPLE 1: FUNCTION AS PARAMETER

```
// Here, op is a function type (Int, Int) -> Int
fun calculator(a: Int,  b: Int,  op: (Int, Int) -> Int): Int {
    return op(a, b)
}

// its calling
val sum = calculator(3,  4,  { x, y -> x + y })
println(sum)  // Output: 7
```

```
//You can also simplify it using lambda
val sum = calculator(3, 4) { x, y -> x + y }
```

EXAMPLE 2: RETURNING A FUNCTION

```
fun chooseOperation(type: String): (Int, Int) -> Int {
    return if (type == "add") { a, b -> a + b } else { a, b -> a * b }
}

val operation = chooseOperation("add")
println(operation(2, 3))  // Output: 5
```

Kotlin Syntax Features For HOFs

- ### Lambda Expressions

Lambda expressions in Kotlin are anonymous functions defined using curly braces. They can take parameters and return values like named functions. Lambdas are essential for defining short, one-off functions and are the building blocks of higher-order functions.

Example:

```
val add = { a: Int, b: Int -> a + b }
println(add(2, 3))  // Output: 5

// With type inference
val multiply: (Int, Int) -> Int = { x, y -> x * y }
```

- ### Trailing Lambdas

When the last parameter of a function is a lambda, Kotlin allows moving the lambda outside of the parentheses. This makes the code more readable and is commonly used in Kotlin's DSLs and collection APIs.

Example:

```
// Traditional way
compute(3, 4, { a, b -> a * b })

// Trailing lambda syntax
compute(3, 4) { a, b -> a * b }
```

- ### `it` Keyword

`it` is the implicit name given to the single parameter of a lambda in Kotlin. This makes simple lambda expressions even more concise by avoiding the need to explicitly name the parameter.

Example:

```
val numbers = listOf(1, 2, 3, 4)
val evens = numbers.filter { it % 2 == 0 }
println(evens)  // Output: [2, 4]
```

- ## Function References

Kotlin allows referencing existing functions using the double colon `::` operator. These references can be passed as arguments to higher-order functions, allowing code reuse and avoiding redundant lambdas.

Example:

```
fun square(x: Int): Int = x * x
val op: (Int) -> Int = ::square
println(op(4))  // Output: 16

val doubled = listOf(1, 2, 3).map(::square)
```

- ## Lambda with Receiver

Lambdas with receivers allow you to call methods on a receiver object inside the lambda without using a qualifier. These are used heavily in Kotlin DSLs.

Example:

```
val greet: String.() -> String = { "Hello, $this!" }
println("Kotlin".greet())  // Output: Hello, Kotlin!
```

- ## Returning Functions

Kotlin supports returning lambdas from functions, enabling closures. Returned functions can capture and use variables from their defining scope.

Example:

```
fun makeAdder(x: Int): (Int) -> Int = { y -> x + y }
val addFive = makeAdder(5)
println(addFive(3))  // Output: 8
```

Built-in Higher-Order Functions in Kotlin

Kotlin provides a powerful standard library with a rich set of built-in higher-order functions for collection manipulation and functional programming. These functions operate on lists, maps, sets, and other iterable types. They take functions (usually lambdas) as parameters and return new transformed collections.

- ### map()

Transforms each element of a collection using the given function and returns a new list of the results.

Example:

```
val numbers = listOf(1, 2, 3)
val doubled = numbers.map { it * 2 }
println(doubled)  // Output: [2, 4, 6]
```

- ### filter()

Filters the elements that satisfy the given predicate and returns a new list.

Example:

```
val numbers = listOf(1, 2, 3, 4, 5)
val evens = numbers.filter { it % 2 == 0 }
println(evens)  // Output: [2, 4]
```

- ### forEach()

Performs the given action (lambda) on each element. It is used for side effects (like printing), not for creating a new collection.

Example:

```
val names = listOf("Anna", "Ben", "Charlie")
names.forEach { println(it) }
```

- **fold()**

Reduces the collection to a single value by applying the operation from left to right, starting with an initial accumulator value.

Example:

```
val numbers = listOf(1, 2, 3, 4)
val sum = numbers.fold(0) { acc, n -> acc + n }
println(sum)  // Output: 10
```

- **reduce()**

Similar to fold, but uses the first element as the initial value and applies the operation from left to right.

Example:

```
val numbers = listOf(1, 2, 3, 4)
val product = numbers.reduce { acc, n -> acc * n }
println(product)  // Output: 24
```

- **any(), all(), none()**

`any` returns true if at least one element matches the predicate. `all` returns true if all elements match. `none` returns true if no elements match.

Example:

```
val numbers = listOf(1, 2, 3)
println(numbers.any { it > 2 })    // true
println(numbers.all { it > 0 })    // true
println(numbers.none { it < 0 })   // true
```

- ### take() and drop()

`take(n)` returns the first `n` elements. `drop(n)` skips the first `n` elements.

Example:

```
val numbers = listOf(1, 2, 3, 4, 5)
println(numbers.take(3))  // Output: [1, 2, 3]
println(numbers.drop(3))  // Output: [4, 5]
```

Currying in Kotlin

- ### Definition of Currying

Currying is the process of transforming a function that takes multiple arguments into a sequence of functions, each taking a single argument. Instead of calling a function with all parameters at once, you call a series of functions that return other functions.

For example, a function of type (A, B) -> C becomes: A -> (B -> C).

This technique enables function reuse and helps in building more modular and readable code.

- ### How Currying Is Achieved in Kotlin

Kotlin does not have native support for automatic currying like Haskell or Scala, but currying can be implemented manually using higher-order functions. You define a function that returns another function, each taking one argument.

Example:

```
fun multiply(x: Int): (Int) -> Int = { y -> x * y }
val double = multiply(2)
println(double(5))  // Output: 10
```

This simulates a curried function by returning a lambda.

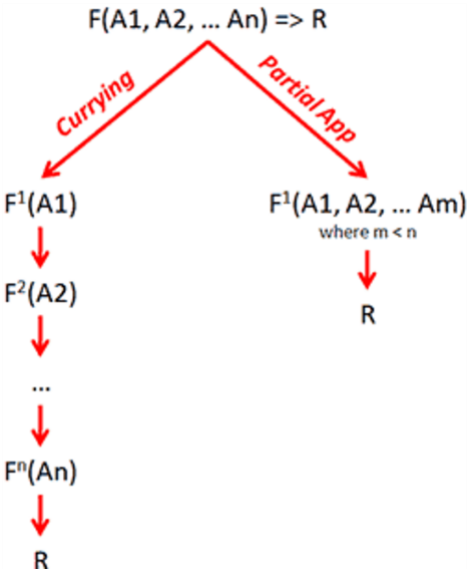
- ## Currying vs Partial Application

Currying transforms a multi-parameter function into a chain of single-parameter functions. Partial application fixes a few arguments of a function and returns a new function with fewer parameters.

Difference:
- Currying always transforms to a series of unary functions.
- Partial application can stop at any stage with any number of arguments fixed.

Example of partial application in Kotlin:

```
fun add(a: Int, b: Int) = a + b
val addFive = { x: Int -> add(5, x) }
println(addFive(3))  // Output: 8
```



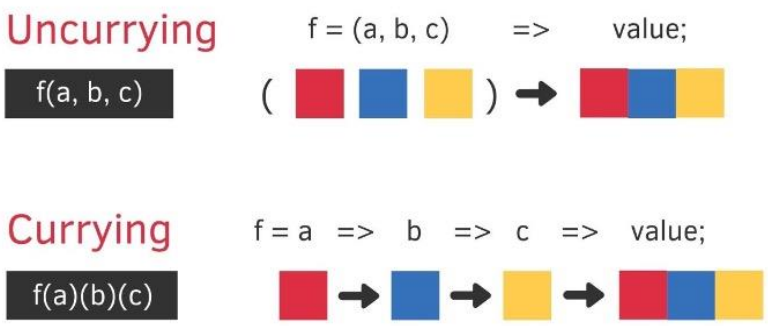
Manual Currying in Kotlin

Manual currying in Kotlin involves writing nested functions or lambdas that take one parameter at a time.

Example:

```
fun curriedSum(a: Int): (Int) -> (Int) -> Int = { b -> { c -> a
+ b + c } }
val result = curriedSum(1)(2)(3)
println(result)  // Output: 6
```

Each invocation returns a new function expecting the next argument.



Use Cases Of HOF & Currying In Kotlin

Use Case	HOFs	Currying
DSLs	Used to build readable config-like code (e.g., Jetpack Compose).	Rarely used.
Callbacks	Used a lot for passing logic in UIs or APIs.	Not used here.
Events	Common for click listeners, etc.	Not practical.
Data Pipelines	Chain functions like map, filter, etc.	Create flexible pipelines with partial values.
Custom Functions	Pass lambdas to reuse logic.	Return functions with pre-filled values.
Delayed Execution	Run lambdas later (e.g., with inline).	Wait until all arguments are supplied.

Scala: HOFs & Currying

HOFs in Scala

Why are HOFs important in Scala?

• Scala is a multi-paradigm language, but embraces functional programming. HOFs make code more concise and composable.
• Using HOFs allows reusing logic with different behaviors and reducing code repetition.
• You can pass in your own function or lambda to dynamically control behavior (e.g., sorting, filtering, reducing).

Function as Parameter

Example 1

```
def applyOperation(x: Int, operation: Int => Int): Int = {
  operation(x)
}
val square = (n: Int) => n * n
val result = applyOperation(5, square)
println(result)  // Output: 25

// You can also pass the lambda directly
val result2 = applyOperation(6, x => x + 10)
println(result2)  // Output: 16
```

Example 2

```
def calculate(a: Int, b: Int, op: (Int, Int) => Int): Int = {
  op(a, b)
}
val sum = calculate(3, 4, (x, y) => x + y)
val product = calculate(3, 4, _ * _)
println(s"Sum: $sum")  // Output: Sum: 7
println(s"Product: $product")  // Output: Product: 12
```

Returning a Function

Example 1

```
def multiplier(factor: Int): Int => Int = {
  (x: Int) => x * factor
}
val triple = multiplier(3)  // triple is a function: Int => Int
println(triple(4))  // Output: 12
```

Example 2

```
def greetWith(greeting: String): String => String = {
  (name: String) => s"$greeting, $name!"
}

val sayHello = greetWith("Hello")

val sayWelcome = greetWith("Welcome")

println(sayHello("Scala"))  // Output: Hello, Scala!

println(sayWelcome("Sidrit"))  // Output: Welcome, Sidrit!
```

Trailing Lambdas in Scala

• When the last parameter of a function is a lambda, Scala allows moving the lambda outside of the parentheses.

Example:

```
object LambdaExample extends App {
  def compute(a:Int, b:Int, op:(Int,Int) => Int):Int = op(a,b)
  // Traditional way
  val result1 = compute(3, 4, (a: Int, b: Int) => a * b)
  println(result1)
  // Trailing lambda syntax
  val result2 = compute(3, 4) { (a: Int, b: Int) => a * b }
  println(result2)
}
```

"_" as a Placeholder In Scala

• When you want to write a simple anonymous function but don't want to explicitly name the parameters, you can use _ as a placeholder.

• Scala will replace each _ with a different parameter in order.

Example:

```
def compute(a: Int, b: Int, op: (Int, Int) => Int): Int = op(a,
b)

// Using named params
 val sum1 = compute(3, 4, (x, y) => x + y)

// Using _ placeholder syntax
 val sum2 = compute(3, 4, _ + _)

println(sum1)  // Output: 7
println(sum2)  // Output: 7
```

Built-in Higher-Order Functions in Scala

1. map()

- Applies a function to each element, returns a new collection of results.

Example:

```
val nums = List(1, 2, 3, 4)
// Using explicit parameter x
val doubled = nums.map(x => x * 2)
// Using explicit parameter x
val doubled = nums.map(_ * 2)
```

2. filter()

- Returns a collection with elements that satisfy a predicate (boolean test).

Example:

```
val nums = List(1, 2, 3, 4)

val evens = nums.filter(_ % 2 == 0)  // List(2, 4)
```

3. flatmap()

- Like map, but the function returns a collection which gets flattened.

Example:

```
val words = List("hello", "world")
val chars = words.flatMap(_.toList)
// List('h','e','l','l','o','w','o','r','l','d')
```

4. reduce()

- Combines elements using a binary function to produce a single value.

Example:

```
val nums = List(1, 2, 3, 4)
val sum = nums.reduce(_ + _)  // 10
```

5. fold()

- Similar to reduce but allows specifying a starting (initial) value.

Example:

```
val nums = List(1, 2, 3)
val sum = nums.fold(0)( _ + _ )  // 6
```

6. foreach()

- Applies a function to each element, typically for side effects (returns Unit).

Example:

```
val nums = List(1, 2, 3)
nums.foreach(println) // 1 2 3 (in new lines)
```

7. exists() and forall()

- exists checks if any element satisfies a predicate.
- forall checks if all elements satisfy a predicate.

Example:

```
val nums = List(1, 2, 3, 4)
nums.exists(_ > 3)  // true
nums.forall(_ > 0)  // true
```

8. sortBy()

- Sort elements by a function or natural ordering.

```
val words = List("apple", "banana", "pear")
val sortedWords = words.sortBy(_.length) // List("pear", "apple", "banana")
```

Currying in Scala

Definition of Currying

• Currying is the process of transforming a function that takes multiple arguments into a sequence of functions, each taking a single argument.

• In other words:

```
// Normal function: takes 2 parameters
def add(x: Int, y: Int): Int = x + y
// Curried version: a function that returns another function
def curriedAdd(x: Int)(y: Int): Int = x + y
```

Why use currying in Scala?

• Partial application: You can fix some arguments and create a new function.

• Function composition: It fits well with functional programming patterns.

• Cleaner syntax: Especially when passing functions as arguments.

How is Currying Achieved in Scala?

1. Using Multiple Parameter Lists

• This is the most direct way to define a curried function:

```
def multiply(x: Int)(y: Int): Int = x * y
val timesTwo = multiply(2)_  // Partial application
println(timesTwo(5))  // Output: 10
```

2. Using the curried Method

• You can convert a regular function into a curried one.

```
val add = (x: Int, y: Int) => x + y
val curriedAdd = add.curried
val addFive = curriedAdd(5)
println(addFive(3))  // Output: 8
```

Manual Currying in Scala

We start with a regular function:

```
val add = (x: Int, y: Int) => x + y
```

Then we manually curry it:

```
def manualCurriedAdd(x: Int): Int => Int = {
  (y: Int) => x + y
}
```

Equivalent to this:

```
def curriedAdd(x: Int)(y: Int): Int = x + y
```

• manualCurriedAdd takes an Int and returns a function Int => Int.

• You call manualCurriedAdd(5) to get a function that adds 5 to whatever you give it.

Why do we use Currying in Scala?

1. Partial Application & Impoved Code Reuse

•You can fix some parameters and create a new, more specific function.

```
def multiply(x: Int)(y: Int): Int = x * y
val double = multiply(2) _  // Partially applied
println(double(5))  // Output: 10
```

2. Better Function Composition

•Curried functions work well with higher-order functions and pipelines.

```
val add = (x: Int) => (y: Int) => x + y
val add2 = add(2)
val result = List(1, 2, 3).map(add2)  // List(3, 4, 5)
```

Where is Currying Mostly Used?

1. Functional Programming

Currying is a core part of functional programming in Scala, Haskell, etc.

2. Frameworks like Play Framework (Scala Web)

In Play Framework, controller actions often use currying to inject dependencies like request handlers or authentication layers.

3. DSLs and APIs

Libraries that build domain-specific languages (DSLs), like Akka HTTP or Cats, often use currying for fluent API design.

4. Partial Functions

Currying makes it easier to create partially applied functions for use in higher-order functions like map, filter, etc.

Language Comparisons

Comparison of HOFs in Python, Kotlin, and Scala

Aspect	Python	Kotlin	Scala
Function as First-Class	Yes, functions can be assigned and passed around.	Yes, functions are first-class citizens.	Yes, core FP concept. Everything is an object including functions.
Lambda Support	Yes, but single-expression lambdas only.	Yes, supports inline multi-parameter lambdas.	Yes, full-featured anonymous functions and blocks.
Built-in HOFs	map, filter, reduce (via functools), sorted, etc.	map, filter, fold, forEach, etc. in standard library.	map, filter, fold, reduce, etc. on collections and Streams.
Function Reference	Use function name without () or use functools.partial.	Use ::functionName syntax.	Pass function names directly, no special syntax needed.
Lambdas with Receivers	Not supported directly.	Yes, supports receivers for DSLs.	Supported using implicit classes or context.

Comparison of Currying in Python, Kotlin, and Scala

Aspect	Python	Kotlin	Scala
Native Support	No native currying. Can be done manually or with functools.partial.	No native support, but can be done manually via nested lambdas.	Yes, supports native currying using multiple parameter lists.
Syntax Simplicity	Manual via nested functions or lambda chains.	Requires explicit nested functions or lambdas.	Simple and elegant syntax: def add(a: Int)(b: Int).
Partial Application	Supported using functools.partial.	Manual only using inline lambdas.	Fully supported as a first-class feature.
Use in Practice	Rare in Python codebases.	Occasional for configuration or closures.	Common in functional-style Scala code.

Summary

This project explored two foundational concepts in functional programming—Higher-Order Functions (HOFs) and Currying—across three modern languages: **Python**, **Kotlin**, and **Scala**. Through detailed theoretical explanations, practical examples, and syntax demonstrations, we analyzed how each language supports these concepts and how they contribute to code reusability, modularity, and functional composition.

- **Python** supports HOFs natively and enables currying through closures and the functools.partial utility.
- **Kotlin** integrates HOFs deeply into its standard library and allows manual currying via nested lambdas.
- **Scala** offers the most complete support with native syntax for both HOFs and currying, embracing functional paradigms fully.

By comparing syntax features, real-world applications, and functional strengths, this study highlights the value of these paradigms in writing clean, maintainable, and expressive code. Overall, the project demonstrates the power of treating functions as first-class citizens and the flexibility that currying provides in building reusable logic across various domains.